

Verifying Claims in Code Generation Research: A Methodological Audit of G3, GRPO, and Ensemble Strategies Against Established Benchmarks

Deconstruction and Verification of the Core Comparative Framework: G3, GRPO, and Multi-Arm Ensembling

The central task of this audit is to conduct a rigorous verification of the comparative claims made within the research paper concerning three distinct methodologies: G3, GRPO, and multi-arm ensembling. This section undertakes a foundational deconstruction of each term, leveraging the provided body of literature to establish their definitions, applications, and contextual relevance. The primary objective is to ascertain whether these components are well-defined concepts within the existing scientific discourse and to identify any ambiguities or potential contradictions that could undermine the paper's validity. The analysis reveals that while GRPO is a clearly established reinforcement learning algorithm, the acronyms "G3" and "multi-arm ensembling" appear to be either novel contributions or non-standard terminologies requiring careful scrutiny.

GRPO, or Group Relative Policy Optimization, emerges from the provided sources as a highly specialized and well-documented reinforcement learning (RL) algorithm [28](#). Its core purpose is to enhance the stability and performance of large language model (LLM) training, particularly for tasks demanding complex reasoning [26](#) [28](#). Unlike traditional RL algorithms such as PPO that rely on a separate critic model to estimate the value of actions, GRPO achieves stabilization by normalizing rewards within a small group of parallel rollouts generated for a single prompt [27](#) [125](#). This group-normalized advantage estimation makes it computationally efficient and well-suited for optimizing policy learning across large-scale models [30](#). The algorithm has been prominently featured in recent research focused on improving the coding abilities of LLMs, with applications ranging from enhancing code correctness to improving notions of quality, maintainability, and security [120203](#). Its success has catalyzed significant research interest, leading to

numerous variants and extensions designed for multi-turn interactions, self-correction, and collaborative agent systems [79](#) [82](#) [168](#)[196](#). The algorithm is frequently cited as a state-of-the-art approach for post-training reasoning models [168](#)[170](#). Therefore, the paper's use of GRPO as a baseline or component of its proposed framework is methodologically sound and aligns with contemporary best practices in the field. Any factual error in this context would not relate to the viability of GRPO itself but rather to a misrepresentation of its mechanics or an implausible claim about its performance relative to other methods. For instance, the paper must correctly describe the training loop, including how groups of responses are sampled and rewarded, and whether it adheres to implementation-specific requirements, such as the need for a prefix-preserving chat template in certain frameworks [52](#).

In stark contrast to the clarity surrounding GRPO, the acronym "G3" presents a significant point of ambiguity. An exhaustive review of the provided source materials reveals at least two distinct and unrelated uses of this term. In one context, a research paper proposes "G3" as a novel framework based on Retrieval-Augmented Generation (RAG) for worldwide geo-alignment [178](#). In another, the same acronym appears in the title of a different paper, "Graph Preflexor-GRPO," suggesting it might be part of a model name or identifier related to a Programming Knowledge Graph (PKG) framework [49](#) [179](#). Crucially, none of the provided sources offer a definition for the specific "G3" framework proposed in the target paper under review. This creates a critical knowledge gap that precludes any meaningful comparison until the acronym is explicitly defined within the paper itself. Without this definition, "G3" remains an undefined entity, rendering any comparative analysis involving it purely speculative. It could stand for a proprietary model architecture, a novel integration of technologies (e.g., Graph-based Reasoning, GRPO, and another component), or even refer to a specific experimental setup. The first and most essential step in the auditing process is to locate and formally define "G3" upon its first appearance in the manuscript. Only with a clear understanding of what "G3" represents can a reviewer assess its claims, its relationship to GRPO, and its purported advantages or disadvantages.

The third component of the comparative framework, "multi-arm ensembling," also lacks a direct citation as a standard, widely recognized term in the provided literature. However, its conceptual foundation is logically consistent with several active and well-established research paradigms. The term itself suggests a strategy that involves aggregating the outputs or capabilities of multiple, distinct components—referred to as "arms"—to improve overall system performance, robustness, or reliability. This concept aligns closely with the broader trend of using ensemble methods in LLM research [183](#). Such methods aim to mitigate the uncertainty inherent in single-model predictions and often yield

superior performance compared to individual models 70 72 . The word "arm" is particularly suggestive of multi-agent systems (MAS), where different agents collaborate or specialize in distinct roles 74 81 . For example, the Stronger-MAS framework introduces an agent- and turn-wise grouped RL algorithm (AT-GRPO) to manage interactions among multiple agents 200, while ReMA leverages multi-agent reinforcement learning to elicit meta-thinking in LLMs 127. Similarly, mixture-of-experts (MoE) architectures represent a form of dynamic ensembling, where different "expert" sub-networks are activated for different inputs 75 . The paper's proposal of "multi-arm ensembling" may therefore be a novel application or a rebranding of an existing ensemble technique. The key to verifying this claim lies entirely within the paper's own description. The reviewer must determine if this approach constitutes a genuinely new contribution or simply a specific instantiation of a known principle. To be verifiable, the paper must provide a clear architectural diagram and a step-by-step explanation of the ensembling mechanism. For instance, it must specify whether the "arms" are independent models, different layers of a single model, or specialized heads within a larger network. It must also detail the aggregation strategy: are the final outputs selected based on a voting mechanism, combined through logit averaging, or managed through a collaborative dialogue structure? Without these specifics, the claim remains a high-level abstraction lacking empirical substance.

Acronym/ Concept	Definition & Context in Provided Literature	Status for Auditing
GRPO	Group Relative Policy Optimization is a stable, on-policy RL algorithm used for fine-tuning LLMs for reasoning tasks, including code generation 26 28 120. It normalizes rewards within a group of samples instead of using a critic model 27 .	Well-Established. Its use is credible and methodologically sound. The audit should focus on implementation details and reported performance claims.
G3	Used in two distinct ways: 1) A RAG-based framework for geo-alignment 178; 2) Part of a model name in a graph-based reasoning paper 179. No definition is available for the specific "G3" in the target paper.	Highly Ambiguous. The primary finding of this analysis. The exact meaning of "G3" in the target paper must be obtained directly from its text before any further comparison can be made.
Multi-Arm Ensembling	Not a standard term found in the provided sources. The concept is logically consistent with established areas like LLM ensembles 70 183, multi-agent systems 74 81 , and mixture-of-experts architectures 75 .	Plausible but Unverified. The novelty and specific implementation depend entirely on the paper's internal description. The audit must assess whether it is a genuine innovation or a rebranding of an existing technique.

In summary, the audit of the core comparative framework begins with a clear hierarchy of confidence. GRPO serves as a solid, verifiable baseline. The terms "G3" and "multi-arm ensembling" are more problematic, representing points of significant uncertainty. The review's success hinges on resolving the ambiguity of "G3" and obtaining a sufficiently detailed description of the ensembling mechanism to evaluate its novelty and

effectiveness. Until these definitional gaps are filled, any quantitative comparisons reported in the supplementary CSV files cannot be meaningfully assessed.

Analysis of Methodological Choices: LoRA, Graph Prefix Tokens, and Their Synergies

Beyond the high-level comparison of G3, GRPO, and multi-arm ensembling, a deep methodological audit requires a granular examination of the specific techniques employed within the paper. The user specifically highlighted LoRA configuration and the use of graph prefix tokens as key areas of interest. This section provides a detailed analysis of these two methodological choices, drawing upon the provided literature to understand their standard implementations, potential synergies, and known challenges. The analysis uncovers a significant and well-documented tension between the combination of LoRA and GRPO, which represents a critical area for scrutiny. Furthermore, the concept of "graph prefix tokens," while plausible, remains a high-level abstraction that requires a clear, justified definition from the source paper to be considered a valid technical claim.

Low-Rank Adaptation (LoRA) is a dominant technique for the parameter-efficient fine-tuning of large pre-trained models [43](#) [158](#). Instead of updating the billions of parameters in the original model weights, LoRA freezes the base model and injects trainable low-rank matrices into the layers of the transformer architecture, most commonly the query (Q) and value (V) projection matrices in the attention mechanism [56](#) [116](#). This approach dramatically reduces the number of trainable parameters, cutting memory consumption and training time by up to 75% and 90%, respectively, while keeping the original base model untouched [47](#). The effectiveness of LoRA is highly sensitive to its hyperparameters, primarily the rank (r) and the alpha (α) scaling factor [113](#) [117](#). Best practices suggest starting with smaller ranks, such as $r=8$ and $\alpha=16$, and tuning these values based on validation set performance [162](#). Some studies indicate that for domain adaptation, smaller ranks may be sufficient, whereas higher ranks might be needed for more complex tasks [161](#) [162](#). The paper under review must report these specific hyperparameters to allow for external verification and reproducibility. LoRA is not only used for general fine-tuning but has also been integrated into more complex architectures, such as deep ensemble methods (LoRA-Ensemble) for improved uncertainty modeling [70](#) [72](#) and mixture-of-experts (LoRA-MoE) systems to balance expert utilization [75](#).

The most significant finding from the provided literature concerning LoRA is its documented instability when combined with the GRPO algorithm. Multiple sources explicitly warn of severe negative side effects when enabling LoRA during GRPO training. One GitHub issue reports that when training with GRPO and LoRA enabled, the model's generated output "becomes completely garbled" and collapses into meaningless text [53](#). Another similar issue describes the output becoming "乱的重复的" (garbled and repetitive) after merging the LoRA weights [164](#). A survey paper further corroborates this, stating that combining LoRA with GRPO can lead to poor policy stability [102](#). This is a critical red flag because GRPO is a core component of the paper's proposed framework. The existence of this known conflict implies that a successful implementation is non-trivial and requires specific engineering solutions. The literature suggests potential mitigations, such as carefully designing the reward function to guide the learning process [75](#) or controlling the LoRA update formula, $\Delta W = (A \times B) \times (\alpha / r)$, where A and B are the low-rank matrices being learned [55](#). A responsible research paper would not merely state "we trained with GRPO+LoRA" but would include a dedicated subsection explaining the specific measures taken to overcome this documented instability. The absence of such a discussion would constitute a major methodological omission and cast serious doubt on the validity of the reported results. The reviewer must scrutinize the paper for any mention of this challenge and the strategies employed to address it.

The second key methodological choice, "graph prefix tokens," is a plausible but abstract concept. Given the extensive research on integrating graph neural networks (GNNs) with LLMs, the idea of converting graph structural information into a textual format for the LLM is highly feasible [58](#) [60](#) [63](#). The term "prefix token" strongly suggests a technique where this graph-derived information is prepended to the input prompt, serving as a structured context for the model. The literature supports several mechanisms for achieving this. For example, natural language can be used to describe a graph, effectively creating a textual representation [61](#). More technically, graphs can be converted into token sequences; one approach involves creating a prefix tree where unique paths from the root to a leaf correspond to language-based IDs for nodes [59](#). Other methods involve mapping node embeddings produced by a GNN directly as tokens for the LLM [64](#) or using graph convolutional networks to model syntactic dependency graphs and integrate them into the LLM's attention mechanism [103](#)[105](#). The paper's claim is only verifiable if it provides a precise and unambiguous description of its "graph prefix token" generation process. It must specify: What type of graph is being used (e.g., Abstract Syntax Tree, dependency graph)? What algorithm converts the graph into a sequence of tokens? Are the tokens human-readable identifiers, or are they embeddings mapped into the LLM's vocabulary? Without these details, the claim remains a high-level description of an idea rather than a concrete, implementable method. The plausibility of the concept is high,

but its validity depends entirely on the rigor of its implementation and documentation as presented in the paper.

Methodological Component	Standard Implementation & Best Practices	Potential Issues / Conflicts	Requirement for Verifiability
LoRA (Low-Rank Adaptation)	Injects trainable low-rank matrices, typically into Q and V attention layers 56 116. Hyperparameters rank and alpha are crucial and should be tuned 113162.	Generally stable, but can suffer from poor policy stability when combined with GRPO 102. Output collapse into garbled text has been reported 53 164.	Must report all key hyperparameters (r, alpha, target modules). Must discuss how GRPO+LoRA instability was addressed.
Graph Prefix Tokens	Plausible technique for providing structured graph context. Methods include converting graphs to natural language descriptions 61 or token sequences via traversal paths 59 . Can be integrated via attention masks 103 or token embedding 64 .	No direct evidence of conflict, but the effectiveness is highly dependent on the quality of the graph-to-text/token conversion.	Must provide a clear, step-by-step algorithm for generating the tokens from the input graph. Must justify the chosen conversion method.

In conclusion, the methodological audit reveals a critical tension between the paper's use of GRPO and LoRA. While both are individually valid techniques, their combination is fraught with documented challenges that demand explicit justification from the authors. The reviewer must treat this combination as a high-risk claim. Simultaneously, the concept of "graph prefix tokens" is technically plausible but remains an unverified abstraction without a clear, detailed description of its implementation. The paper must bridge these gaps by providing exhaustive methodological details to ensure its claims are scientifically sound and reproducible.

External Benchmarking and Performance Metric Verification

A cornerstone of the research goal is the external factual verification of the paper's numerical claims, particularly the performance comparisons between G3, GRPO, and multi-arm ensembling. This requires placing the reported metrics, such as CodeBLEU and pass@k, within the evolving landscape of code generation benchmarks. The provided literature offers a wealth of external data points and contextual insights that are essential for this verification process. The analysis reveals that the significance of any reported performance gain is critically dependent on the choice of benchmark, as older benchmarks have become saturated, making frontier models nearly indistinguishable from one another. The emergence of more challenging benchmarks is a crucial trend that must inform the interpretation of the paper's results.

The primary metrics for evaluating code generation models are pass@k and CodeBLEU [84](#) [85](#). Pass@k measures the functional correctness of generated code by checking if at least one of k generated solutions passes a suite of automated tests for a given problem [130](#). Common variants include pass@1, pass@5, and pass@10, allowing for a comparison of a model's relative advantages and disadvantages [84](#). CodeBLEU, on the other hand, is a metric that extends the traditional BLEU score to measure the syntactic similarity between generated code and reference solutions, rewarding shared n-grams in the code [195](#). Both metrics are standard tools in the field, and the user's confirmation that they are used in the paper is consistent with established practice [130](#)[134](#).

However, interpreting these metrics requires a deep understanding of the benchmarks on which they are measured. The literature indicates a significant and accelerating phenomenon known as "benchmark saturation." Traditional benchmarks like HumanEval and MBPP, which were once the gold standard for evaluating LLMs, have become increasingly ineffective at discriminating between different models [15](#). As of mid-2026, frontier models achieve near-perfect scores on HumanEval, with pass@1 scores reaching 96-99% [15](#) [139](#). For instance, top models on the Codesota leaderboard from July 2026 include Claude Sonnet 4.5 with a 97.6% pass@1 score, followed closely by R1 at 97.4% [18](#). Similarly, on MBPP, top models reach scores around 94.2% pass@1 [139](#). These benchmarks were originally designed to test the ability to write small, self-contained Python functions [17](#). While achieving high scores here demonstrates proficiency, it tells us little about a model's ability to tackle more complex, real-world programming tasks that require deeper reasoning, tool use, or environment awareness [17](#). The rapid progress of models like DeepSeek-R1 and others has rendered these benchmarks largely obsolete for cutting-edge research [149](#)[168](#).

This saturation has spurred the development of more challenging and realistic benchmarks. The introduction of HumanEval Pro and MBPP Pro represents a direct attempt to create harder versions of the classic benchmarks by framing problems as "self-invoking" tasks, where a model must solve a basic problem first and then use that solution to solve a more complex, related problem [32](#) [33](#) [209](#). These new benchmarks reveal significant performance gaps between models that were previously hidden on the original datasets [92](#) [94](#). Beyond this, a new generation of benchmarks has emerged to test a wider range of skills. SWE-bench, for example, evaluates models on their ability to resolve actual issues from GitHub repositories, a task that requires reading comprehension, repository navigation, and complex debugging [89](#). LiveCodeBench focuses on live-coding scenarios, while EvalPlus uses more rigorous testing procedures [13](#) [31](#). Open-source models have shown strong performance on these new benchmarks; for

example, one 3B parameter model achieved an 83.5% pass@1 on HumanEval and a notable 62.9 avg@8 on LiveCodeBench v5, outperforming much larger frontier models 132133. When reviewing the paper, it is imperative to know which benchmarks were used. A reported pass@10 score on HumanEval of, say, 80%, might seem mediocre next to the 98% saturation level, but the same score on LiveCodeBench v5 could represent a state-of-the-art achievement. The context of the benchmark is paramount.

The following table synthesizes external benchmark data from the provided sources, offering a snapshot of the competitive landscape for code generation models as of 2024-2026.

Benchmark	Top Model(s)	Reported Score	Date/Context
HumanEval	Claude Sonnet 4.5	97.6% pass@1 18	July 2026
HumanEval	Various Frontier Models	96-99% pass@1 15 139	2023-2026
MBPP	Unknown	94.2% pass@1 139	2025-2026
LiveCodeBench v5	X-Coder model series	62.9% avg@8 132	2025
SWE-bench	Claude Sonnet 3.5	81.7% win rate 13	June 2024
Bigcodebench-Lite Pro	LLMs show consistent trends	Generalizability emphasized 31	2024
HumanEval Pro / MBPP Pro	New benchmarks introduced	Show performance gaps vs. original benchmarks 92	2024

For the audit, the reviewer must obtain the supplementary CSV files and cross-reference the numbers against this external data. The review should ask: 1. **Benchmark Choice:** Did the paper use a modern, challenging benchmark (like HumanEval Pro, LiveCodeBench, or SWE-bench) if it claims to present a state-of-the-art framework? Using only HumanEval or MBPP would severely limit the perceived impact of its findings. 2. **Score Interpretation:** What is the absolute and relative significance of the reported CodeBLEU and pass@k scores? A marginal improvement in pass@10 on HumanEval might not be a meaningful result, whereas a lower pass@1 on a difficult benchmark could be. 3. **Consistency with Trends:** Do the reported scores align with the general performance trends observed in the literature? For example, do the scores for the GRPO baseline align with what is known about GRPO's typical performance gains? Are the scores for the proposed G3 and multi-arm ensembling methods plausibly better or worse than the baselines?

In essence, external verification is not just about checking if a number is correct, but about assessing its significance. The rapidly evolving nature of benchmarks means that a factual claim about a model's performance is meaningless without the proper context.

The reviewer's task is to place the paper's results on this evolving timeline and judge them accordingly.

Internal Consistency and Reproducibility Assessment

A comprehensive factual error review must extend beyond external verification to an internal consistency check, ensuring that the paper's narrative, methodology, and results form a coherent and logically sound whole. This dimension of the audit focuses on the principles of scientific reproducibility and logical coherence. It demands that the authors provide sufficient detail to allow an independent researcher to replicate their experiments and that the claimed results follow logically from the described methods. Key aspects of this assessment include verifying the logical flow of the methodology, ensuring all critical hyperparameters are disclosed, and confirming that qualitative observations align with quantitative metrics.

The most fundamental requirement for internal consistency is the logical connection between the proposed methods and the reported outcomes. The paper should tell a clear story: "We hypothesized that [G3/multi-arm ensembling] would improve code generation over a standard GRPO baseline. To test this, we implemented [detailed methodology], trained the models on [dataset/benchmark], and evaluated them using [metrics]. Our results show [quantitative findings], which qualitatively demonstrate [qualitative analysis]." Each part of this narrative must be supported by the text. For instance, if the paper claims that its "graph prefix tokens" method provides superior performance, the qualitative analysis of the generated code should highlight examples where the structural information from the graph led to more correct or efficient solutions. Conversely, if the paper reports a failure mode, such as increased logical errors, the qualitative analysis should provide representative examples of these failures. A disconnect between the quantitative results and the qualitative claims would be a major red flag. For example, reporting a high CodeBLEU score but providing generated code that is syntactically nonsensical would indicate a problem with either the metric calculation or the model's output.

Reproducibility is a bedrock principle of scientific research, and its absence is a form of factual error. The paper must disclose all critical hyperparameters for every component of its experimental setup. For the LoRA configurations, this includes the rank, α , the learning rate, the target modules (e.g., which linear layers in the transformer were adapted), and the number of training epochs [45](#) [46](#) . For the GRPO implementation,

essential hyperparameters include the group size (the number of parallel rollouts per prompt), the learning rate, the KL divergence regularization settings (which help prevent catastrophic forgetting), and the specific reward shaping strategy [123125](#). The paper should also specify the base model architecture (e.g., Qwen2.5-Coder-7B, Llama), the dataset used for training and/or evaluation, and the hardware configuration if it is a key part of the contribution (e.g., single GPU training). The lack of these details renders the work non-reproducible and makes it impossible for the community to validate the claims. Several sources emphasize the importance of such details, providing guides on LoRA hyperparameter tuning [113114](#) and documenting GRPO trainer configurations [188190](#). The target paper's failure to meet this standard would be a significant methodological flaw.

Furthermore, the internal consistency check must examine the described interplay between the different components of the proposed framework. The paper combines GRPO, LoRA, and a graph-based method. The interaction between GRPO and LoRA was already identified as a potential source of instability [53 102](#). The paper must have a section discussing how this was managed. Similarly, it must explain how the "graph prefix tokens" are fed into the model that is simultaneously being fine-tuned with GRPO and LoRA. Is the graph information processed by a separate encoder before being concatenated with the main input? Or is it directly embedded into the tokenizer's vocabulary? The paper needs to provide a clear architectural overview or diagram that illustrates this entire pipeline. Without such a diagram and accompanying text, it is impossible to verify if the described system is a simple sequential chain or a more complex, intertwined architecture. Sources discussing multi-encoder fusion systems [76](#) and graph-integrated LLMs [104106](#) provide examples of the kind of architectural detail expected.

Finally, the internal logic should account for the scale of the models being used. The literature distinguishes between Large Language Models (LLMs) and Small Language Models (SLMs), with some researchers considering models below 10 billion parameters to be SLMs as of 2025 [144](#). The computational cost and training dynamics can differ significantly between these categories. The paper should situate its work within this context. If it is training a 7B parameter model on a single consumer GPU, this is a notable contribution [40](#). If it is training a 100B parameter model, the computational resources required would be substantial. The choice of model size should be justified in relation to the complexity of the task and the proposed methods. For example, using a very large model for a task that could be solved by a smaller one might raise questions about efficiency, while using a small model for a highly complex task might explain a lower performance ceiling.

In summary, the internal consistency and reproducibility assessment acts as a final gatekeeper for the paper's credibility. The reviewer must meticulously check for:

- **Logical Flow:** Do the methods lead to the results, and do the results support the conclusions?
- **Qualitative-Quantitative Alignment:** Do the examples provided in the text match the statistical performance reported?
- **Hyperparameter Disclosure:** Are all critical settings for LoRA, GRPO, and the base model fully and transparently reported?
- **Architectural Clarity:** Is there a clear explanation of how all components (GRPO, LoRA, graph) interact within the system?
- **Model Contextualization:** Is the choice of model size justified and appropriately framed?

Any failure in these areas weakens the paper's argument and undermines the factual validity of its claims, regardless of how impressive the final numbers in the CSV file might appear.

Synthesis of Findings and Final Recommendations for the Review

This comprehensive audit has systematically deconstructed and analyzed the components of the research paper, focusing on the verification of its claims comparing G3, GRPO, and multi-arm ensembling. The review has examined both the external validity of its performance metrics against established benchmarks and the internal consistency of its methodological choices. The findings reveal a mixed picture, with some elements grounded in well-established research while others present significant areas of ambiguity and potential contradiction that require careful resolution. The following synthesis consolidates the key findings and provides actionable recommendations for completing the fact-checking process.

The most critical finding of this audit is the profound ambiguity surrounding the acronym "G3." The analysis of the provided literature shows that "G3" is not a universally recognized term in the field of code generation and has been used to denote at least two different, unrelated concepts [178179](#). Consequently, the first and most urgent recommendation is to locate and formally define "G3" within the target paper. Without this definition, any comparative analysis involving G3 is premature and fundamentally

flawed. The reviewer must determine if "G3" is a novel acronym for the paper's proposed framework, a specific identifier for a model variant, or a shorthand for a combination of technologies. This clarification is the single most important prerequisite for any further evaluation.

Secondly, the paper's methodological choice to combine the GRPO reinforcement learning algorithm with Low-Rank Adaptation (LoRA) for fine-tuning represents a high-risk claim. The provided sources contain multiple reports of severe instability, including model output collapsing into garbled text, when these two techniques are used together [53](#) [164](#). While LoRA is a standard and effective technique for parameter-efficient tuning [43](#), and GRPO is a state-of-the-art algorithm for training reasoning models [26](#), their union is not straightforward. The paper must dedicate a specific section to addressing this known conflict. It should explicitly discuss the challenges encountered and detail the mitigation strategies employed, such as specific reward shaping [75](#) or careful control of the LoRA update formula [55](#). If the paper fails to acknowledge or address this significant pitfall, the reported results should be viewed with extreme skepticism, as the performance gains could be an artifact of unstable training rather than a genuine improvement in the model's capabilities.

Third, the external verification of the paper's performance claims is contingent on the context of the benchmarks used. The widespread saturation of traditional benchmarks like HumanEval and MBPP, where frontier models now achieve near-perfect scores of 96-99% pass@1, makes them poor discriminators of incremental progress [15](#) [18](#) [139](#). The reviewer must use the supplementary CSV files not just to check for arithmetic accuracy but to assess the *significance* of the reported gains. Performance on newer, more challenging benchmarks like HumanEval Pro, LiveCodeBench, or SWE-bench is far more indicative of a model's true capabilities and the novelty of its approach [33](#) [89](#) [132](#). The reviewer should compare the paper's scores against the external data points summarized in this report to gauge whether the results represent a marginal improvement or a substantial leap forward.

Finally, the two novel technical claims—the use of "graph prefix tokens" and the implementation of "multi-arm ensembling"—must be held to a high standard of specificity. While both concepts are plausible and align with broader trends in LLM research, they remain abstract ideas without a clear, unambiguous description from the paper. The paper must provide a step-by-step algorithm for generating the graph prefix tokens from a given graph structure and must detail the exact mechanism of the multi-arm ensembling process, including how the individual "arms" are defined and how their

outputs are aggregated. Furthermore, the paper must report all critical hyperparameters for both LoRA and GRPO to ensure the work is reproducible.

In conclusion, the path to a definitive verdict on the paper's factual accuracy is clear. The reviewer must proceed by first defining "G3," then critically examining the paper's handling of the GRPO+LoRA conflict, contextualizing the reported performance metrics against modern benchmarks, and demanding specific, verifiable details for its novel technical components. By following these steps, the audit can move from identifying areas of concern to delivering a conclusive and evidence-based assessment of the paper's claims.

Reference

1. Improve Code Generation by Pass@k-Maximized Code Ranking <https://arxiv.org/html/2408.05715v1>
2. [PDF] CODEJUDGE : Evaluating Code Generation with Large Language ... <https://aclanthology.org/2024.emnlp-main.1118.pdf>
3. [PDF] CodeGeeX: A Pre-Trained Model for Code Generation with ... <https://keg.cs.tsinghua.edu.cn/jietang/publications/KDD23-Zheng-CodeGeeX.pdf>
4. xjywhu/Awesome-Multimodal-LLM-for-Code - GitHub <https://github.com/xjywhu/Awesome-Multimodal-LLM-for-Code>
5. CodeGeeX: A Pre-Trained Model for Code Generation with ... https://www.researchgate.net/publication/372910674_CodeGeeX_A_Pre-Trained_Model_for_Code_Generation_with_Multilingual_Benchmarking_on_HumanEval-X
6. [PDF] Evaluating Code Generation by Large Language Models <https://www.diva-portal.org/smash/get/diva2:1985844/FULLTEXT01.pdf>
7. [PDF] MULTI-LINGUAL EVALUATION OF CODE GENERATION MODELS <https://cogcomp.seas.upenn.edu/papers/AGWLTTAWSSGDKFFJGQRNRBDRX23.pdf>
8. CodeScore: Evaluating Code Generation by Learning Code Execution <https://www.semanticscholar.org/paper/CodeScore%3A-Evaluating-Code-Generation-by-Learning-Dong-Ding/a95817df43516f5ff9cfc447209def1fcd569afc>
9. [PDF] Evaluating Large Language Models in Class-Level Code Generation <https://mingwei-liu.github.io/assets/pdf/ICSE2024ClassEval-V2.pdf>

10. [PDF] Framework for evaluating code generation ability of ... - ETRI KSP <https://ksp.etri.re.kr/ksp/article/file/68695.pdf>
11. HumanEval & MBPP Benchmark Leaderboard 2025–2026 - Codesota <https://codesota.com/llm/humaneval-mbpp>
12. [PDF] Evaluating Large Language Models on Self-invoking Code Generation <https://aclanthology.org/2025.findings-acl.686.pdf>
13. EvalPlus Leaderboard <https://evalplus.github.io/leaderboard.html>
14. Evaluating Large Language Models on Self-invoking Code Generation <https://arxiv.org/html/2412.21199v1>
15. HumanEval, MBPP, and LiveCodeBench - Code Benchmarks in 2026 <https://benchmarkingagents.com/humaneval/>
16. [PDF] Evolving Coding Benchmarks via LLM - OpenReview <https://openreview.net/pdf?id=zZa7Ke7WAJ>
17. HumanEval and MBPP: What a Code Benchmark Won't Tell You <https://verityai.co/blog/humaneval-mbpp-code-generation-benchmarks>
18. HumanEval Leaderboard 2026 - Compare AI Model Scores <https://pricepertoken.com/leaderboards/benchmark/humaneval>
19. Towards Evaluating the Code Intelligence of LLMs From Multiple ... <https://www.computer.org/csdl/proceedings-article/apsec/2025/665300b013/2emR0AquZXi>
20. Big Code Models Leaderboard - Smolagents <https://smolagents.org/big-code-models-leaderboard/>
21. Posterior-GRPO: Rewarding Reasoning Processes in Code ... - arXiv <https://arxiv.org/html/2508.05170v1>
22. [PDF] GRPO-MA: MULTI-ANSWER GENERATION IN GRPO - OpenReview <https://openreview.net/pdf/5bb799637a0d71b12d91def27530d42d57b25861.pdf>
23. [PDF] MEML-GRPO: Heterogeneous Multi-Expert Mutual Learning for ... <https://ojs.aaai.org/index.php/AAAI/article/view/40391/44352>
24. GRPO++: Tricks for Making RL Actually Work <https://cameronrwolfe.substack.com/p/grpo-tricks>
25. Daily Papers - Hugging Face <https://huggingface.co/papers?q=S-GRPO>
26. Training reasoning models with GRPO algorithm - Facebook <https://www.facebook.com/groups/DeepNetGroup/posts/2600497447009788/>
27. GitHub - junfanz1/GRPO: Search-R1 fine-tunes LLMs to decide ... <https://github.com/junfanz1/GRPO>

28. Training a Multi Language 3B Coder LM with Reinforcement ... <https://medium.com/@yogeshsingla481/training-a-multi-language-3b-coder-lm-with-reinforcement-learning-grpo-f37577d5e5f7>
29. (PDF) M-GRPO: Stabilizing Self-Supervised Reinforcement ... https://www.researchgate.net/publication/398720344_M-GRPO_Stabilizing_Self-Supervised_Reinforcement_Learning_for_Large_Language_Models_with_Momentum-Anchored_Policy_Optimization
30. GRPO Algorithm Overview - Emergent Mind <https://www.emergentmind.com/topics/grpo-algorithm>
31. Evaluating Large Language Models on Self-invoking Code Generation <https://arxiv.org/html/2412.21199v2>
32. HumanEval Pro and MBPP Pro: Evaluating Large Language Models ... <https://answers111.github.io/evalpro.github.io/>
33. Evaluating Large Language Models on Self-invoking Code Generation <https://www.semanticscholar.org/paper/HumanEval-Pro-and-MBPP-Pro%3A-Evaluating-Large-Models-Yu-Zhao/44c47a0bf21d0b555e7aedc1cd8a9bbf3295d46d>
34. Evaluating Large Language Models on Self-invoking Code Generation <https://liner.com/review/humaneval-pro-and-mbpp-pro-evaluating-large-language-models-on>
35. Evaluating Large Language Models on Self-invoking Code Generation <https://huggingface.co/papers/2412.21199>
36. Evaluating Large Language Models on Self-invoking Code Generation https://www.researchgate.net/publication/387539984_HumanEval_Pro_and_MBPP_Pro_Evaluating_Large_Language_Models_on_Self-invoking_Code_Generation
37. CodeEval-Pro HumanEval Pro and MBPP Pro: Evaluating ... - GitHub <https://github.com/CodeEval-Pro/CodeEval-Pro>
38. Practical Tips for Finetuning LLMs Using LoRA (Low-Rank Adaptation) <https://magazine.sebastianraschka.com/p/practical-tips-for-finetuning-llms>
39. Exploring Parameter-Efficient Fine-Tuning Techniques for Code ... <https://arxiv.org/html/2308.10462v3>
40. LoRA & QLoRA Fine-Tuning: Build Custom LLMs on a Single GPU ... <https://www.meta-intelligence.tech/en/insight-lora-finetuning>
41. LoRAGen: Structure-Aware Weight Space Learning for LoRA ... <https://github.com/tsinghua-fib-lab/LoRAGen>
42. A Survey on Large Language Models for Code Generation <https://dl.acm.org/doi/10.1145/3747588>
43. Easily Train a Specialized LLM: PEFT, LoRA, QLoRA, LLaMA ... <https://cameronrwolfe.substack.com/p/easily-train-a-specialized-llm-peft>

44. Top LoRA Fine-Tuning LLMs Techniques Roundup - Newline <https://www.newline.co/@Dipen/top-lora-fine-tuning-llms-techniques-roundup--007605c4>
45. LoRA fine-tuning Hyperparameters Guide | Unsloth Documentation <https://unsloth.ai/docs/get-started/fine-tuning-llms-guide/lora-hyperparameters-guide>
46. Efficient Fine-Tuning with LoRA for LLMs | Databricks Blog <https://www.databricks.com/blog/efficient-fine-tuning-lora-guide-llms>
47. Fine-Tuning LLMs with LoRA: 2025 Guide - Amir Teymoori <https://amirteymoori.com/fine-tuning-llms-with-lora-a-practical-guide-for-2025/>
48. GitHub - OpenRLHF/OpenRLHF: An Easy-to-use, Scalable and High ... <https://github.com/openrlhf/openrlhf>
49. lamm-mit/graph-preflexor-grpo · GitHub <https://github.com/lamm-mit/graph-preflexor-grpo>
50. grpo-flat with zero dataset - GitHub <https://github.com/XU-YIJIE/grpo-flat>
51. [RFC] Prefix-Tree Shared Attention for Multi-Turn RL Training #6401 <https://github.com/verl-project/verl/issues/6401>
52. GRPO Trainer - Hugging Face https://huggingface.co/docs/trl/grpo_trainer
53. Model output collapses into garbled text during GRPO + LoRA + ... <https://github.com/verl-project/verl/issues/3226>
54. Pref-GRPO: Pairwise Preference Reward-based GRPO for Stable ... <https://github.com/CodeGoat24/Pref-GRPO>
55. Training for Reasoning with GRPO — part II (a step by ... - Medium <https://medium.com/@lucamassaron/training-for-reasoning-with-grpo-part-ii-a-step-by-step-explanation-f80c219e2059>
56. GRPO Demo - Tunix documentation https://tunix.readthedocs.io/en/latest/_collections/examples/grpo_gemma.html
57. Discussion about GRPO training setup for Qwen-Image-Edit-2509 or ... <https://github.com/X-GenGroup/Flow-Factory/discussions/52>
58. A Survey of Large Language Models for Graphs - arXiv <https://arxiv.org/html/2405.08011v1>
59. [PDF] LLM AS GNN: GRAPH VOCABULARY LEARNING ... - OpenReview <https://openreview.net/pdf?id=1FY1apsMxc>
60. Large Language Models Meet Graph Neural Networks - MDPI <https://www.mdpi.com/2227-7390/13/7/1147>
61. [PDF] Each graph is a new language: Graph Learning with LLMs <https://aclanthology.org/2025.findings-acl.902.pdf>

62. xkLi-Allen/Awesome-GNN-in-LLMs-Papers - GitHub <https://github.com/xkLi-Allen/Awesome-GNN-in-LLMs-Papers>
63. [PDF] A Survey of Graph Meets Large Language Model - IJCAI <https://www.ijcai.org/proceedings/2024/0898.pdf>
64. Graph Neural Networks meet Large Language Models <https://n1o.github.io/posts/graph-neural-networks-meet-large-language-models/>
65. Large Language Models vs Graph Neural Networks: It Depends <https://www.symmetry-systems.com/blog/large-language-models-vs-graph-neural-networks-it-depends/>
66. [PDF] GRAPH NEURAL NETWORKS FOR RELATIONSHIP EXTRACTION ... <https://upcommons.upc.edu/bitstreams/d353cfe4-6032-413b-b7c5-447390037a8e/download>
67. A Gentle Introduction to Graph Neural Networks - Distill.pub <https://distill.pub/2021/gnn-intro>
68. The Engineering Handbook for GRPO + LoRA with Verl <https://huggingface.co/blog/Weyaxi/engineering-handbook-grpo-lora-with-verl>
69. RL(HF) algorithms with LoRA Support - verl documentation https://verl.readthedocs.io/en/latest/advance/ppo_lora.html
70. LoRA-Ensemble: Efficient Uncertainty Modelling for Self-attention ... <https://arxiv.org/html/2405.14438v3>
71. Align, Don't Divide: Revisiting the LoRA Architecture in Multi-Task... <https://openreview.net/forum?id=Mz98kwANpF>
72. LoRA-Ensemble: Efficient Uncertainty Modelling for Self-attention ... <https://github.com/prs-eth/lora-ensemble>
73. [PDF] Randomized Multi-Head LoRA for Efficient Multi-Task Learning <https://aclanthology.org/2025.findings-emnlp.35.pdf>
74. [PDF] RAAS: LLM Agentic System Architecture Search with GRPO https://openaccess.thecvf.com/content/CVPR2026/papers/Yang_RAAS_LLM_Agentic_System_Architecture_Search_with_GRPO_CVPR_2026_paper.pdf
75. Unlocking LoRA-MoE for GRPO via Mechanism-Aware Rewards <https://cs.nju.edu.cn/lm/en/publication/ma-balancing-2026/index.html>
76. [PDF] multi-encoder fusion with lora and rank normalization for first-shot https://dcase.community/documents/challenge2026/technical_reports/DCASE2026_Balozi_28_t2.pdf
77. Collaborative Reinforcement Learning for Code Generation - arXiv <https://arxiv.org/html/2605.24812v1>

78. [PDF] Reinforcement Learning for Safe LLM Code Generation - EECS <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2025/Archive/EECS-2025-123.pdf>
79. [PDF] MURPHY: Reflective Multi-Turn Reinforcement Learning for Self ... <https://cdn.amazon.science/e9/ad/ea154c21428eb99d228d8cc55fb2/murphy-forlm-final.pdf>
80. Enhancing Code LLMs with Reinforcement Learning in ... - Qeios <https://www.qeios.com/read/8G8TB2>
81. On-Policy Reinforcement Learning for Collaborative LLMs <https://huggingface.co/papers/2510.11062>
82. MURPHY: Multi-Turn GRPO for Self Correcting Code Generation https://www.researchgate.net/publication/397521667_MURPHY_Multi-Turn_GRPO_for_Self_Correcting_Code_Generation
83. Multistep Reasoning Agents (with GRPO & RLEF) - Project Euler ... <https://blog.ando.ai/posts/ai-grpo/>
84. A Survey on Evaluating Large Language Models in Code ... - arXiv <https://arxiv.org/html/2408.16498v2>
85. (PDF) A Survey on Evaluating Large Language Models in Code ... https://www.researchgate.net/publication/383529947_A_Survey_on_Evaluating_Large_Language_Models_in_Code_Generation_Tasks
86. Improve Code Generation with Feedback | OpenReview <https://openreview.net/forum?id=CscKx97jBi>
87. xlang-paper-reading/llm-code-generation.md at main - GitHub <https://github.com/clang-ai/clang-paper-reading/blob/main/llm-code-generation.md>
88. Using Large Language Models for Aerospace Code Generation <https://www.mdpi.com/2226-4310/12/6/498>
89. SWE-bench Leaderboard 2026: Top Models Ranked + What It ... <https://localaimaster.com/models/swe-bench-explained-ai-benchmarks>
90. GitHub - openai/human-eval: Code for the paper "Evaluating Large ... <https://github.com/openai/human-eval>
91. HumanEval Pro 和MBPP Pro: 在自引代码生成上评估大型语言模型 <https://www.chatpaper.ai/zh/paper/8df98e6a-3a99-49e8-9da3-c950e897a5e9>
92. HumanEval Pro and MBPP Pro Evaluation - Emergent Mind <https://api.emergentmind.com/papers/2412.21199>
93. Evaluating Large Language Models on Self-invoking Code Generation <https://arxiv.org/abs/2412.21199>

94. Evaluating Large Language Models on Self-invoking Code Generation <https://www.themoonlight.io/de/review/humaneval-pro-and-mbpp-pro-evaluating-large-language-models-on-self-invoking-code-generation>
95. FareedKhan-dev/multi-agent-training-grpo - GitHub <https://github.com/FareedKhan-dev/multi-agent-training-grpo>
96. Qwen GRPO Graph Extraction RL Finetune - GitHub <https://github.com/wey-gu/grpo-graph-extraction>
97. Qwen3_(4B)-GRPO.ipynb - Colab [https://colab.research.google.com/github/unslothai/notebooks/blob/main/nb/Qwen3_\(4B\)-GRPO.ipynb](https://colab.research.google.com/github/unslothai/notebooks/blob/main/nb/Qwen3_(4B)-GRPO.ipynb)
98. Fine-tuning Llama, using GRPO and LoRa, on a single A100. - Seth https://www.sethweidman.com/odsc_east_2025_grpo_llama
99. LLM Fine-Tuning 2026: LoRA, QLoRA, DPO, GRPO - Future AGI <https://futureagi.com/blog/llm-fine-tuning-guide-2025/>
100. [PDF] GRPO-Guided Modality Selection Enhanced LoRA-Tuned LLMs for ... <https://aclanthology.org/2025.findings-emnlp.1059.pdf>
101. Gated Integration of Low-Rank Adaptation for Continual Learning of ... <https://neurips.cc/virtual/2025/poster/116274>
102. [PDF] Connecting Low-Rank Adapters and Policy Stability in GRPO Fine ... <https://www.merl.com/publications/docs/TR2026-092.pdf>
103. A Graph-Integrated Large Language Model for Repository-Level ... <https://arxiv.org/html/2505.16901v4>
104. [PDF] A Graph-Integrated Large Language Model for Repository-Level ... <https://openreview.net/pdf?id=b98ODdeYq5>
105. Daily Papers - Hugging Face <https://huggingface.co/papers?q=statement-level%20dependency%20graph>
106. [PDF] Code Graph Model (CGM) - arXiv <https://arxiv.org/pdf/2505.16901>
107. Nations of the Americas Chapter of the Association ... - ACL Anthology <https://aclanthology.org/events/naacl-2025/>
108. Program2025 – IEEE CAI 2025 <https://cai.ieee.org/2025/program2025/>
109. Spring 2025 C-Day Program - Kennesaw State University <https://campus.kennesaw.edu/colleges-departments/ccse/events/computing-showcase/sp25-cday-program.php>
110. Appl. Sci., Volume 15, Issue 19 (October-1 2025) – 559 articles <https://www.mdpi.com/2076-3417/15/19>
111. A Survey of Vibe Coding with Large Language Models - arXiv <https://arxiv.org/html/2510.12399v1>

112. From LLMs to LLM-based Agents for Software Engineering - arXiv <https://arxiv.org/html/2408.02479v2>
113. LoRA Hyperparameters: Rank, Alpha & Target Module Selection <https://mbrenndoerfer.com/writing/lora-hyperparameters-rank-alpha-target-modules>
114. Mastering Low-Rank Adaptation (LoRA): The Ultimate Guide to ... <https://prasunmaity.medium.com/mastering-lora-the-ultimate-guide-to-efficient-llm-fine-tuning-5c9de67e7fe2>
115. LoRA - Hugging Face https://huggingface.co/docs/peft/main/conceptual_guides/lora
116. Ultimate Guide to LoRA for LLM Optimization - Newline <https://www.newline.co/@zaoyang/ultimate-guide-to-lora-for-llm-optimization--2ef04eb9>
117. LoRA Hyperparameters: The Tuning Guide - PythonAlchemist <https://www.pythonalchemist.com/blog/lora-hyperparameters-guide>
118. LoRA - Hugging Face https://huggingface.co/docs/peft/package_reference/lora
119. Implementing LoRA from Scratch - Medium <https://medium.com/data-science/implementing-lora-from-scratch-20f838b046f1>
120. Improving LLM-Generated Code Quality with GRPO - OpenReview <https://openreview.net/forum?id=wdJT3KNCv6>
121. GRPO Does Not Close the Multi-Agent Coordination Gap - arXiv <https://arxiv.org/html/2606.07845v1>
122. Maestro: Reinforcement Learning to Orchestrate Hierarchical Model ... <https://arxiv.org/html/2605.22177v1>
123. Weight Ensembling Improves Reasoning in Language Models - arXiv <https://arxiv.org/html/2504.10478v1>
124. Reasoning Beyond Limits: Advances and Open Problems for LLMs <https://arxiv.org/html/2503.22732v1>
125. Selective Rollout: Mid-Trajectory Termination for Multi-Sample Agent ... <https://arxiv.org/html/2605.05802v1>
126. CaP-X: A Framework for Benchmarking and Improving Coding ... <https://arxiv.org/html/2603.22435v1>
127. [PDF] ReMA: Learning to Meta-think for LLMs with Multi-agent ... - NIPS <https://openreview.net/pdf/4ef88667f9d6e59455e6805cd91df2e8e519f9d0.pdf>
128. The Landscape of Agentic Reinforcement Learning for LLMs: A Survey <https://arxiv.org/html/2509.02547v1>
129. Review of Inference-Time Scaling Strategies: Reasoning, Search ... <https://arxiv.org/html/2510.10787v1>

130. claude_code__claude-opus-4-6__max__enabled.jsonl https://huggingface.co/datasets/clem/mentionsanalysis_whatsbest_april26/blob/main/claude_code__claude-opus-4-6__max__enabled.jsonl
131. Daily Papers - Hugging Face <https://huggingface.co/papers?q=code%20sanitization>
132. Daily Papers - Hugging Face <https://huggingface.co/papers?q=codebase-level%20reasoning>
133. Daily Papers - Hugging Face <https://huggingface.co/papers?q=quantum%20software%20development%20framework>
134. Daily Papers - Hugging Face <https://huggingface.co/papers?q=Code%20clone%20detection>
135. Daily Papers - Hugging Face <https://huggingface.co/papers?q=backend%20refinement>
136. Daily Papers - Hugging Face <https://huggingface.co/papers?q=golden%20RTL%20implementations>
137. Daily Papers - Hugging Face <https://huggingface.co/papers?q=client-side%20encoder>
138. Daily Papers - Hugging Face <https://huggingface.co/papers?q=exact-match%20evaluation>
139. Daily Papers - Hugging Face <https://huggingface.co/papers?q=coding-agent%20benchmarks>
140. LLM-Based Multi-Agent Systems for Code Generation - arXiv <https://arxiv.org/html/2604.16321v1>
141. Environment-Aware Code Generation: How far are We? - arXiv <https://arxiv.org/html/2601.12262v1>
142. Generating Energy-Efficient Code via Large-Language Models - arXiv <https://arxiv.org/html/2509.10099v1>
143. Lego: An LLM Skill-Based Front-End Design Generation Platform <https://arxiv.org/html/2604.23355v1>
144. [PDF] Small Language Models are the Future of Agentic AI - arXiv <https://arxiv.org/pdf/2506.02153>
145. [PDF] TRAINING LARGE LANGUAGE MODELS TO REASON IN ... <https://openreview.net/pdf/7b9893fba01b9b621abc8d6114afb5cdf3afb9da.pdf>
146. Can Large Language Models Help Students Prove Software ... - arXiv <https://arxiv.org/html/2506.22370v1>
147. Interpretable and Verifiable Hardware Generation with LLM-Driven ... <https://arxiv.org/html/2606.19387v1>

148. DesignAsCode: Bridging Structural Editability and Visual Fidelity in ... <https://arxiv.org/html/2602.17690v1>
149. [PDF] AUTOCODE: LLMs AS PROBLEM SETTERS FOR COM <https://openreview.net/pdf?id=F96nsbbhXC>
150. GRPO Guided Adapter Sparsity Policy for Cross Lingual Transfer <https://arxiv.org/abs/2601.06702>
151. 8 - Hugging Face https://huggingface.co/helboukkouri/character-bert-medical/resolve/30397d827839963ebdd7260716f2e92308bdf1f5/mlm_vocab.txt?download=true
152. vocab.txt - Hugging Face <https://huggingface.co/cointegrated/LaBSE-en-ru/resolve/c487ba6fafa76f43d71aeda77ebaff91d75b1e91/vocab.txt?download=true>
153. Conferencia | Cyberhades <https://www.cyberhades.com/categories/conferencia>
154. SLICE CHEESE: Mac OS X で Movable Type 日本語化パッチ <http://kaeruheaven.under.jp/slicecheese/archives/000149.html>
155. <https://huggingface.co/ambroad/bert-multilingual-...> <https://huggingface.co/ambroad/bert-multilingual-passage-reranking-msmarco/commit/e0bde5a695ff8719787ff2f01cc9c58fbffea7f9.diff?file=vocab.txt>
156. Introduction to Low Rank Approximation (LoRA) - Medium <https://medium.com/@mujahidabdullahi1992/an-introduction-to-lora-unpacking-the-theory-and-practical-implementation-e665c5d78295>
157. How to Implement LoRA with Hugging Face PEFT - YouTube https://www.youtube.com/watch?v=F29Y_y7g6IY
158. Low-rank Adaptation of Large Language Models - Nexla <https://nexla.com/enterprise-ai/low-rank-adaptation-of-large-language-models/>
159. A comprehensive overview of everything I know about fine-tuning. https://www.reddit.com/r/LocalLLaMA/comments/1ilkamr/a_comprehensive_overview_of_everything_i_know/
160. What is Lora Fine Tuning? The Definitive Guide - Truefoundry <https://www.truefoundry.com/blog/lora-fine-tuning>
161. Practical Tips for Finetuning LLMs Using LoRA (Low-Rank Adaptation) <https://magazine.sebastianraschka.com/p/practical-tips-for-finetuning-llms/comments>
162. How to Tune LoRA Hyperparameters for Efficient LLMs - LinkedIn https://www.linkedin.com/posts/naveen-kumar-dsn_lora-llm-generativeai-activity-7379120358806896640-5QIo
163. avnlp/llm-finetuning: Advanced LLM fine-tuning techniques ... - GitHub <https://github.com/avnlp/llm-finetuning>

164. grpo训练中, rollout的prompt和completions是正常的, 但是把lora ... <https://github.com/modelscope/ms-swift/issues/9256>
165. AMAP-ML/Tree-GRPO: [ICLR 2026] Tree Search for LLM Agent ... <https://github.com/AMAP-ML/Tree-GRPO>
166. HumanEval Pro and MBPP Pro: Evaluating Large Language Models ... <https://dblp.org/rec/conf/acl/Yu0CZ25>
167. [https://huggingface.co/Thang203/topic_model_general...](https://huggingface.co/Thang203/topic_model_general_normal_april8/commit/c2041e78b0ea380eff4624f329fe5ba616f70f29.diff?file=ctfidf_config.json) https://huggingface.co/Thang203/topic_model_general_normal_april8/commit/c2041e78b0ea380eff4624f329fe5ba616f70f29.diff?file=ctfidf_config.json
168. The Landscape of Agentic Reinforcement Learning for LLMs: A Survey <https://arxiv.org/html/2509.02547v5>
169. StealthRL: Reinforcement Learning Paraphrase Attacks for Multi ... <https://arxiv.org/html/2602.08934>
170. LLM Post-Training: A Deep Dive into Reasoning Large Language ... <https://arxiv.org/html/2502.21321v2>
171. [PDF] RLVR-World: Training World Models with Reinforcement Learning <https://openreview.net/pdf/4b0d2935ed4554453743ecdcf099e0d679355328.pdf>
172. Reward Modeling for Reinforcement Learning-Based LLM Reasoning <https://arxiv.org/html/2602.09305v2>
173. Dynamic Policy Optimization for Multi-Turn Interactive Reasoning <https://openreview.net/forum?id=OWDBiMKYdo>
174. Energy-Aware Code Generation with LLMs: Benchmarking Small vs ... <https://arxiv.org/html/2508.08332v1>
175. [PDF] A framework for assessing the capabilities of code generation ... - arXiv <https://arxiv.org/pdf/2603.05278>
176. Can LLM Replace Stack Overflow? A Study on Robustness and ... <https://arxiv.org/html/2308.10335v5>
177. [PDF] DynaCode: A Dynamic Complexity-Aware Code Benchmark ... - arXiv <https://arxiv.org/pdf/2503.10452>
178. G3: An Effective and Adaptive Framework for Worldwide ... - arXiv <https://arxiv.org/abs/2405.14702>
179. The Navigation Paradox in Large-Context Agentic Coding: Graph ... <https://arxiv.org/html/2602.20048v1>
180. [PDF] arXiv:2308.10335v5 [cs.CL] 27 Jan 2024 <https://arxiv.org/pdf/2308.10335>
181. Context-Based Adversarial Attacks on AI Code Generators - arXiv <https://arxiv.org/html/2606.10945v1>

182. zjukg/KoPA: [Paper][ACM MM 2024] Making Large ... - GitHub <https://github.com/zjukg/KoPA>
183. A curated list of Awesome-LLM-Ensemble papers for the ... - GitHub <https://github.com/junchenzhi/Awesome-LLM-Ensemble>
184. The Rise of AI-Native Software Engineering: Implications for Practice ... <https://arxiv.org/html/2606.12986v1>
185. Codified Context: Infrastructure for AI Agents in a Complex Codebase <https://arxiv.org/html/2602.20478v1>
186. F-GRPO: Don't Let Your Policy Learn the Obvious and Forget the Rare <https://openreview.net/forum?id=DWS8VtJt6a>
187. Goekdeniz-Guelmez/mlx-lm-lora: Train Large Language Models on ... <https://github.com/Goekdeniz-Guelmez/mlx-lm-lora>
188. Bharath2/Minimal-GRPO - GitHub <https://github.com/Bharath2/Minimal-GRPO>
189. codelion/ellora: Enhancing LLMs with LoRA - GitHub <https://github.com/codelion/ellora>
190. examples/grpo_trainer · main · Vaibhav DHANUKA / verl - GitLab https://git.doit.wisc.edu/vdhanuka/verl/-/tree/main/examples/grpo_trainer
191. [PDF] Promoting Exploration to Improve Multi-Domain Reasoning - arXiv <https://arxiv.org/pdf/2511.12344>
192. A Survey of Vibe Coding with Large Language Models - arXiv <https://arxiv.org/html/2510.12399v2>
193. Daily Papers - Hugging Face <https://huggingface.co/papers?q=binary%20registers>
194. Daily Papers - Hugging Face <https://huggingface.co/papers?q=Fisher%20dataset>
195. Daily Papers - Hugging Face <https://huggingface.co/papers?q=structure-aware%20generation>
196. Murphy: Multi-Turn GRPO for Self Correcting Code Generation - arXiv <https://arxiv.org/html/2511.07833v1>
197. Posterior-GRPO: Rewarding Reasoning Processes in Code ... - arXiv <https://arxiv.org/html/2508.05170v2>
198. Stronger-MAS: Multi-Agent Reinforcement Learning for Collaborative... <https://openreview.net/forum?id=IdF6JqXWzx>
199. A Multi-Objective GRPO Approach - arXiv <https://arxiv.org/html/2503.21819v1>
200. On-Policy Reinforcement Learning for Collaborative LLMs - arXiv <https://arxiv.org/html/2510.11062v1>
201. MURPHY: Feedback-Aware GRPO with Retrospective Credit ... - arXiv <https://arxiv.org/abs/2511.07833>

202. Improving LLM-Generated Code Quality with GRPO - arXiv <https://arxiv.org/html/2506.02211v1>
203. AlgoForge: Specializing Code Generation Agents through... <https://openreview.net/forum?id=KwqbtKeaRl>
204. Think Anywhere in Code Generation - arXiv <https://arxiv.org/html/2603.29957v1>
205. Multi-Level Multi-Turn RL Outperforms GRPO: Reasoning with ... <https://openreview.net/forum?id=u1RjV99DPu>
206. HumanEval Pro and MBPP Pro: Evaluating Large Language Models ... <https://aclanthology.org/2025.findings-acl.686/>
207. HumanEval Pro and MBPP Pro: Evaluating Large Language Models ... <https://underline.io/events/485/posters/20628/poster/120779-humaneval-pro-and-mbpp-pro-evaluating-large-language-models-on-self-invoking-code-generation-task?tab=poster>
208. self-llm/models/Qwen3/10-Qwen3_8B_GRPO.ipynb at master - GitHub https://github.com/datawhalechina/self-llm/blob/master/models/Qwen3/10-Qwen3_8B_GRPO.ipynb
209. [论文评述] HumanEval Pro and MBPP Pro: Evaluating Large ... <https://www.themoonlight.io/zh/review/humaneval-pro-and-mbpp-pro-evaluating-large-language-models-on-self-invoking-code-generation>